535240175_Cats_VS_Dogs_P6

April 6, 2026

[1]:
```
import os
import shutil
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import random
import plotly.express as px
import scipy as sp
from scipy import ndimage
from shutil import copyfile
from tensorflow.keras.layers import Conv2D, Add, MaxPooling2D, Dense,␣
↪BatchNormalization, Input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import LearningRateScheduler
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1775460637.315272   25734 port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1775460637.357453   25734 cpu_feature_guard.cc:227] This TensorFlow
binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other
operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1775460638.792457   25734 port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

[2]:
```
!pip install -q kaggle
```

[3]:
```
!mkdir ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
mkdir: cannot create directory '/home/kaming/.kaggle': File exists
```

[4]:
```
!kaggle datasets download -d shaunthesheep/microsoft-catsvsdogs-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-
dataset
License(s): other
microsoft-catsvsdogs-dataset.zip: Skipping, found more recently modified local
copy (use --force to force download)
```

[ ]:
```
!unzip -o microsoft-catsvsdogs-dataset.zip
```

[6]:
```
class_names = ['Cat', 'Dog']

cat_dir = './PetImages/Cat'
dog_dir = './PetImages/Dog'
n_cats = len(os.listdir(cat_dir))
n_dogs = len(os.listdir(dog_dir))
n_images = [n_cats, n_dogs]
px.pie(names=class_names, values=n_images)
```

[7]:
```
#Buat Struktur direktor
try:
  base_dir = './cats-v-dogs'
  folders = [
      'training/cats',
      'training/dogs',
      'validation/cats',
      'validation/dogs',
      'test/cats',
      'test/dogs'
  ]

  for folder in folders:
    os.makedirs(os.path.join(base_dir, folder), exist_ok=True)
  print('Direktori berhasil dibuat.')
except OSError:
  print('Error: Gagal membuat direktori.')
```

```
Direktori berhasil dibuat.
```

[8]:
```
os.listdir('./cats-v-dogs')
```

[8]:
```
['test', 'training', 'validation']
```

[9]:
```
#Define data path
CAT_DIR = './PetImages/Cat'
DOG_DIR = './PetImages/Dog'

TRAINING_CATS = './cats-v-dogs/training/cats'
TRAINING_DOGS = './cats-v-dogs/training/dogs'

VALIDATION_CATS = './cats-v-dogs/validation/cats'
VALIDATION_DOGS = './cats-v-dogs/validation/dogs'

TEST_CATS = './cats-v-dogs/test/cats'
TEST_DOGS = './cats-v-dogs/test/dogs'

#Define wether to include test split or not
INCLUDE_TEST = True
```

[10]:
```
def split_data(
    main_dir,
    training_dir,
    validation_dir,
    test_dir=None,
    include_test=False,
    split_size=0.9
):
    """
    Splits the data into train validation and test sets (optional)

    Args:
    main_dir (string): path containing the images
    training_dir (string): path to be used for training
    validation_dir (string): path to be used for validation
    test_dir (string): path to be used for test
    include_test (boolean): whether to include a test split or not
    split_size (float): size of the dataset to be used for training
    """

    files = []
    for file in os.listdir(main_dir):
      if os.path.getsize(os.path.join(main_dir, file)):
         files.append(file) #appends file name to a list

    shuffled_files = random.sample(files, len(files)) #shuffles the list
    split = int(split_size * len(shuffled_files))
    train = shuffled_files[:split]

    # Calculate remaining files for validation and test
    remaining_files = shuffled_files[split:]
```

```
    if include_test:
      split_valid_test = int(len(remaining_files)/2)
      validation = remaining_files[:split_valid_test]
      test = remaining_files[split_valid_test:]
    else:
      validation = remaining_files

    for element in train:
      copyfile(os.path.join(main_dir, element), os.path.join(training_dir,␣
↪element))

    for element in validation:
      copyfile(os.path.join(main_dir, element), os.path.join(validation_dir,␣
↪element))

    if include_test:
      for element in test:
        copyfile(os.path.join(main_dir, element), os.path.join(test_dir,␣
↪element))

    print("Split Sucessful!")
```

[11]:
```
split_data(CAT_DIR,
           TRAINING_CATS,
           VALIDATION_CATS,
           TEST_CATS,
           0.9)
split_data(DOG_DIR,
           TRAINING_DOGS,
           VALIDATION_DOGS,
           TEST_DOGS,
           0.9)
```

```
Split Sucessful!
Split Sucessful!
```

[12]:
```
print(len(os.listdir('./cats-v-dogs/training/cats')))
print(len(os.listdir('./cats-v-dogs/training/dogs')))

print(len(os.listdir('./cats-v-dogs/validation/cats')))
print(len(os.listdir('./cats-v-dogs/validation/dogs')))


print(len(os.listdir('./cats-v-dogs/test/cats')))
print(len(os.listdir('./cats-v-dogs/test/dogs')))
```

```
11250
```

```
11250
625
625
625
625
```

[13]:
```
train_gen =ImageDataGenerator(rescale=1./255.)
validation_gen= ImageDataGenerator(rescale=1./255.)

if INCLUDE_TEST:
  test_gen = ImageDataGenerator(rescale=1./255.)
```

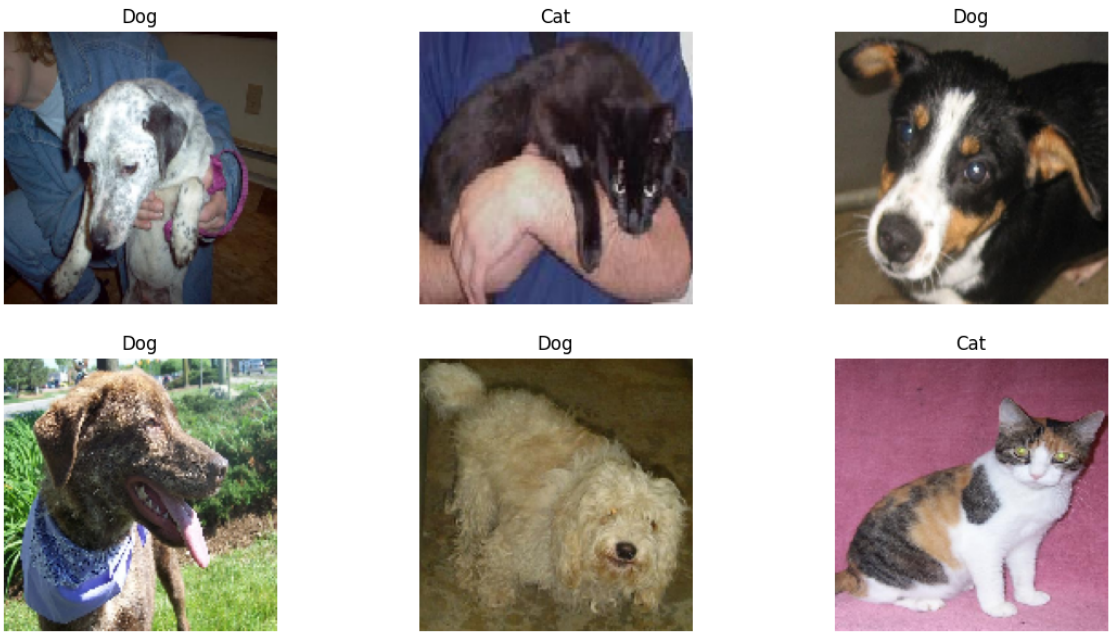
[14]:
```
train_generator = train_gen.flow_from_directory(
    './cats-v-dogs/training',
    target_size=(150, 150),
    batch_size=64,
    class_mode='binary'
)
validation_generator = validation_gen.flow_from_directory(
    './cats-v-dogs/validation',
    target_size=(150, 150),
    batch_size=64,
    class_mode='binary'
)
if INCLUDE_TEST:
  test_generator = test_gen.flow_from_directory(
    './cats-v-dogs/test',
    target_size=(150, 150),
    batch_size=64,
    class_mode='binary'
)
```

```
Found 22498 images belonging to 2 classes.
Found 1250 images belonging to 2 classes.
Found 1250 images belonging to 2 classes.
```

[15]:
```
class_names = ['Cat','Dog']
def plot_data(generator, n_images):
  i = 1
  images, labels = next(generator)
  labels = labels.astype('int32')

  plt.figure(figsize =(14,15))

  for image, label in zip(images, labels):
    plt.subplot(4,3, i)
    plt.imshow(image)
    plt.title(class_names[label])
```

```
    plt.axis('off')
    i += 1
    if i == n_images:
      break

  plt.show()
```

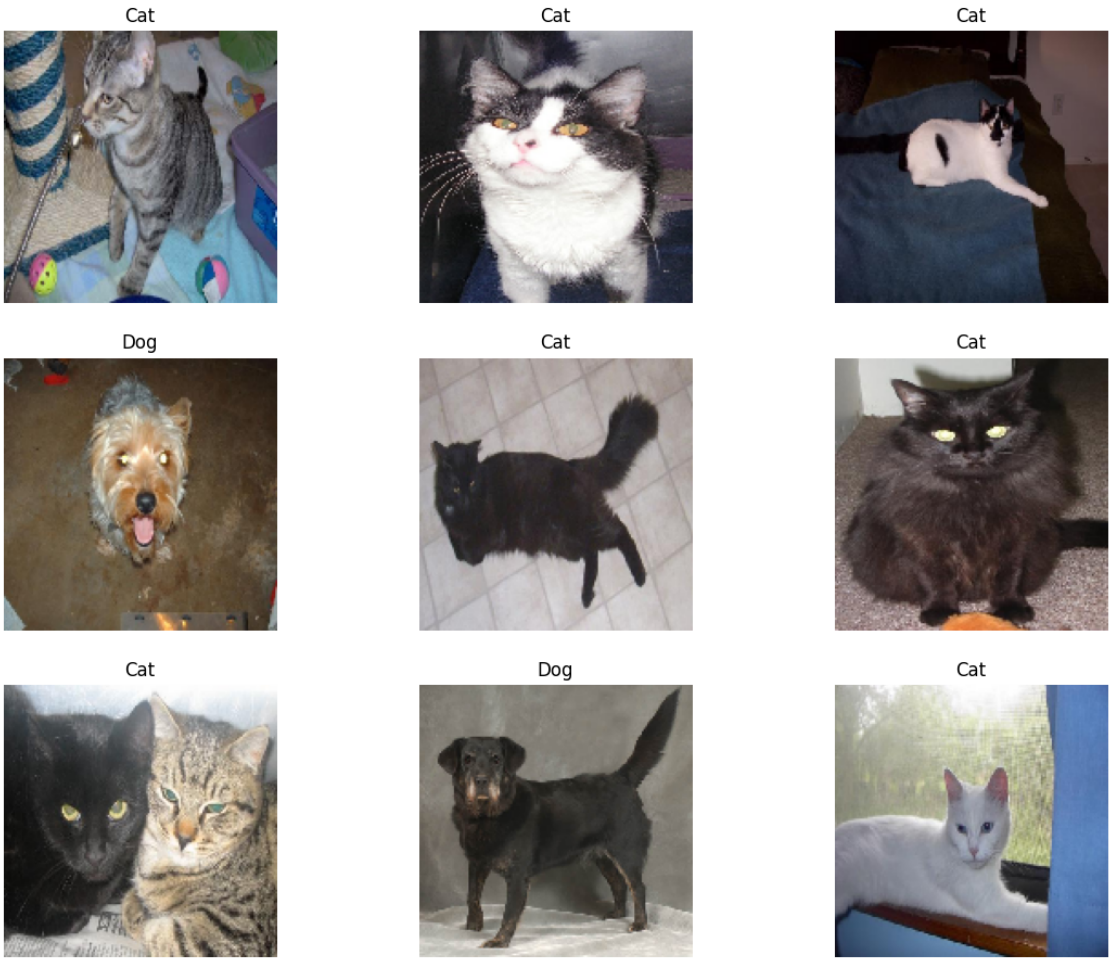
[16]:
```
plot_data(train_generator, 7)
```



[17]:
```
plot_data(validation_generator, 7)
```

[18]:
```
if INCLUDE_TEST:
  plot_data(test_generator, 10)
```

```
/home/kaming/Work/data-science/.venv/lib/python3.12/site-
packages/PIL/TiffImagePlugin.py:949: UserWarning: Truncated File Read
  warnings.warn(str(msg))
```



[19]:
```
#Convolutional Neural Network
inputs = tf.keras.layers.Input(shape=(150,150,3))
x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(inputs)
x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(x)
x = tf.keras.layers.MaxPooling2D(2,2)(x)

x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(x)
x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(x)
x = tf.keras.layers.MaxPooling2D(2,2)(x)

x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(x)
x = tf.keras.layers.Conv2D(16, (3,3), activation='relu')(x)
x = tf.keras.layers.GlobalAveragePooling2D()(x)

x = Dense(32,activation='relu')(x)
x = Dense(16,activation='relu')(x)
x = Dense(16,activation='relu')(x)
```

```
x = tf.keras.layers.Dense(2, activation='softmax')(x)

model = Model(inputs=inputs,outputs=x)
```

```
I0000 00:00:1775460653.198221   25734 gpu_device.cc:2043] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 2610 MB memory:  -> device: 0,
name: NVIDIA GeForce GTX 1650 with Max-Q Design, pci bus id: 0000:01:00.0,
compute capability: 7.5
```

[20]:
```
model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

[21]:
```
r = model.fit(
    train_generator,
    epochs=10,
    validation_data=validation_generator)
```

```
Epoch 1/10

I0000 00:00:1775460654.419220   25734 generator_dataset_op.cc:213] Memory patch
applied: M_TRIM_THRESHOLD=128 kb was set.
I0000 00:00:1775460655.516781   25981 service.cc:153] XLA service 0x7fb7dc035610
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1775460655.516808   25981 service.cc:161]   StreamExecutor [0]:
NVIDIA GeForce GTX 1650 with Max-Q Design, Compute Capability 7.5 (Driver:
13.0.0; Runtime: 12.9.0; Toolkit: 12.5.0; DNN: 9.20.0)
I0000 00:00:1775460655.544507   25981 dump_mlir_util.cc:269] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1775460655.848514   25981 cuda_dnn.cc:461] Loaded cuDNN version
92000
I0000 00:00:1775460655.860837   25981 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_2124__.38
E0000 00:00:1775460656.596327   25981 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1775460659.426798   25981 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1775460661.605793   25981 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.

  2/352              30s 88ms/step - accuracy:
0.5547 - loss: 0.6941

I0000 00:00:1775460663.884418   25981 device_compiler.h:208] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.
```

```
112/352              17s 71ms/step -
accuracy: 0.4870 - loss: 0.6934

I0000 00:00:1775460672.054204   25983 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_2124__.38
E0000 00:00:1775460673.360243   25983 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1775460674.566073   25983 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1775460675.917102   25983 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.

352/352              46s 102ms/step -
accuracy: 0.5332 - loss: 0.6889 - val_accuracy: 0.5232 - val_loss: 0.6808
Epoch 2/10
352/352              27s 75ms/step -
accuracy: 0.6022 - loss: 0.6594 - val_accuracy: 0.6280 - val_loss: 0.6491
Epoch 3/10
352/352              27s 76ms/step -
accuracy: 0.6388 - loss: 0.6316 - val_accuracy: 0.6648 - val_loss: 0.6063
Epoch 4/10
352/352              27s 76ms/step -
accuracy: 0.6740 - loss: 0.6023 - val_accuracy: 0.7256 - val_loss: 0.5626
Epoch 5/10
352/352              26s 75ms/step -
accuracy: 0.6944 - loss: 0.5820 - val_accuracy: 0.7280 - val_loss: 0.5398
Epoch 6/10
352/352              26s 75ms/step -
accuracy: 0.7056 - loss: 0.5685 - val_accuracy: 0.6576 - val_loss: 0.6119
Epoch 7/10
352/352              27s 76ms/step -
accuracy: 0.7155 - loss: 0.5574 - val_accuracy: 0.7264 - val_loss: 0.5486
Epoch 8/10
352/352              26s 75ms/step -
accuracy: 0.7236 - loss: 0.5467 - val_accuracy: 0.7304 - val_loss: 0.5302
Epoch 9/10
352/352              26s 74ms/step -
accuracy: 0.7323 - loss: 0.5399 - val_accuracy: 0.7648 - val_loss: 0.5033
Epoch 10/10
352/352              27s 77ms/step -
accuracy: 0.7357 - loss: 0.5315 - val_accuracy: 0.7536 - val_loss: 0.5041
```

[22]:
```
if INCLUDE_TEST:
  model.evaluate(test_generator)
```

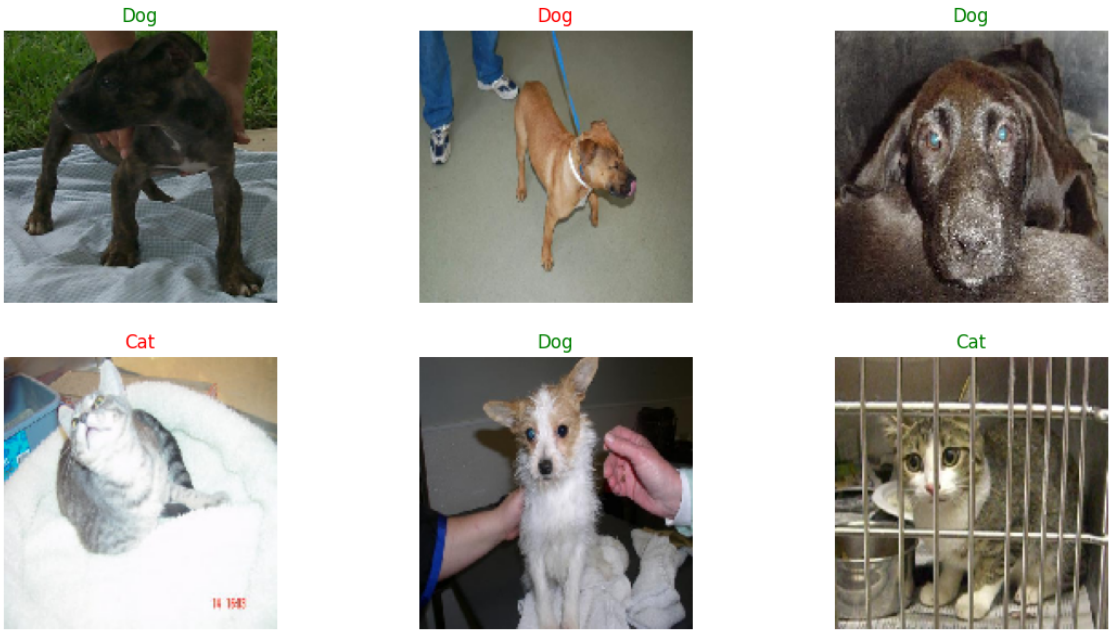
```
20/20              1s 59ms/step -
```

```
accuracy: 0.7208 - loss: 0.5426
```

[23]:
```python
def plot_prediction(generator, n_images):
  """
  Test the modelon random predictions
  Args:
  generator: a generator
  n_images: number of images to plot
  """
  i = 1
  #Get the images and the labels from the generator
  images, labels = next(generator)
  #Gets the model predictions
  preds = model.predict(images)
  predictions = np.argmax(preds, axis=-1)
  labels = labels.astype('int32')
  plt.figure(figsize=(14, 15))
  for image, label in zip(images, labels):
    plt.subplot(4,3,i)
    plt.imshow(image)
    if predictions[i] == labels[i]:
      title_obj = plt.title(class_names[label])
      plt.setp(title_obj, color='g')
      plt.axis('off')
    else:
      title_obj = plt.title(class_names[label])
      plt.setp(title_obj, color='r')
      plt.axis('off')
    i += 1
    if i == n_images:
      break

  plt.show()
```

[24]:
```python
plot_prediction(train_generator, 7)
```

```
2/2              2s 22ms/step
```



[25]:
```
plot_prediction(validation_generator, 7)
```

```
2/2              0s 25ms/step
```

[26]:
```
if INCLUDE_TEST:
    plot_prediction(test_generator, 10)
```

```
2/2              0s 28ms/step
```

4.3 Visualize training histories

[27]:
```
results = pd.DataFrame(r.history)
results.tail()
```

[27]:

	accuracy	loss	val_accuracy	val_loss
5	0.705618	0.568452	0.6576	0.611946
6	0.715530	0.557373	0.7264	0.548586
7	0.723620	0.546745	0.7304	0.530192
8	0.732332	0.539904	0.7648	0.503343
9	0.735665	0.531497	0.7536	0.504052

[28]:
```
fig = px.line(results,
              y=[results['accuracy'], results['val_accuracy']],
              template='seaborn',
              color_discrete_sequence=['#1f77b4', '#ff7f0e'])
fig.update_layout(
    title_font_color='#fad25a',
    xaxis=dict(title='Epochs', color='#fad25a'),
    yaxis=dict(color='#fad25a')
)
fig.show()
```

[29]:
```
fig = px.line(results,
              y=[results['loss'], results['val_loss']],
              template='seaborn',
              color_discrete_sequence=['#ff6b6b', '#4dabf7'])
fig.update_layout(
    title_font_color='#fad25a',
    xaxis=dict(title='Epochs', color='#fad25a'),
    yaxis=dict(color='#fad25a')
)
fig.show()
```